

Machine-Checked Proofs for *The Exact Welfare Guarantee of Fixed-Price Bilateral Trade*

Formalization Supplement

1 What we checked

We formalized the paper’s results in Lean 4 against Mathlib. The accompanying artifact repository contains the Lean sources, the statement manifest and its checker, and the exact and interval-arithmetic certificates for the computer-assisted statements.

The development contains 130 Lean source files and 1,701 theorem or lemma declarations in the library `FixedPrice`. A full build completes 8,605 jobs with no errors. Source scans contain no `sorry`, no custom axiom, and no `native_decide`; every credentialed statement rests only on Lean’s standard foundational axioms `propext`, `Classical.choice`, and `Quot.sound`. The manifest has 101 non-assumption nodes, of which 44 carry a Lean credential. Each of the five lettered theorems and both lettered corollaries has a Lean counterpart among them.

The principal single-unit bounds and optimization results are formalized with no hypothesis beyond the printed ones. The statement comparison in Section 8 identifies clauses not exported by the Lean declarations. Lean proves Theorem A, including the uniqueness of the root C_* , the strict inequality for every pair, and the bounded pairs that approach β_* . It proves the frontier of Theorem B with its conditional form and conjugate duality, the four asymptotic equivalences of Corollary B’, Theorem C with the almost-everywhere uniqueness of the maximizing control, and the dominant-strategy ceiling of Corollary A’ with the price representation of Proposition 12.2. Theorem D is proved by exact evaluation of its two finite instances in the Lean kernel.

The two-unit analysis beyond Theorem D is formalized conditionally. Theorem E, its finite-node recurrence in Appendix B.5, Proposition 13.2, and six of the seven lemmas behind it assume, as named hypotheses, scalar facts, identities and enclosures that exact, interval or SMT arithmetic already certifies (Section 5). The shape lemma, Lemma D.3, needs no such hypothesis.

The manifest records the remaining statements one by one. Proposition 13.1, Proposition D.8, and the reduction of Conjecture 13.3 to the budget bound (66) keep their analytic proofs, and the conjecture itself is open. Lemma 6.1 is formalized in the parts that the proof of Theorem C uses, without its monotone parametrization of the endpoints. From the paper’s Section 3, the arctangent form of $\mathcal{I}$, the existence of a best price, and the fact that randomized prices do no better are not stated in Lean; neither is the two-step formula (85) behind the nonconcavity example of Appendix C.2, whose gap $11/1728$ is checked in exact arithmetic. Decimal values, among them $\beta_* \approx 0.73802$ and the comparison $0.7290804 < 0.738024 < \beta_*$ in Theorem D, come from interval arithmetic outside Lean.

What a credential is attached to. A credential is issued against a pair: the statement of a node, and the set of premises it stands on. The manifest stores a fingerprint of that pair, and the checker recomputes it. Changing a statement or rewiring a premise therefore invalidates the credential. This

check compares the stored statement and premise set with their recorded fingerprint. It does not read the TeX or compare its meaning with a Lean type; that correspondence requires the separate review described in Section 7.

2 The manifest as a graph

The manifest is a directed graph. One vertex represents each statement it tracks, and an edge from P to Q records that the proof of Q uses P . At this revision it has 103 vertices and 164 edges. Thirty-five vertices record the paper itself: 2 assumptions, 4 definitions, and 29 statements. Twenty-four record scalar facts certified by exact, interval or SMT arithmetic, and 44 record Lean renderings of printed results. Forty-one vertices are sources and 31 are sinks. The graph is acyclic, and a manifest containing a cycle fails to load.

A credential is issued against a vertex together with its in-edges. We fingerprint the pair consisting of a node’s statement and its sorted premise set, so the record for Lemma 7.1 reads

```
sha256:2c80e5b31004 = sha256(<statement of P_gaps> |
P_energy,P_preliminaries,P_reference)
```

where the statement is the text of the lemma as printed. Editing the statement moves the fingerprint, and so does adding a premise, dropping one, or rerouting an edge. The next `check` then refuses the credential. Figure 1 draws the in-neighbourhood that Lemma 7.1 is fingerprinted against.

The two reachability queries. Downstream, `blast <node>` returns the descendant closure: what else falls if this statement falls. The two-unit model `A_two_model` reaches 28 of the 103 vertices, and the calculus of the single-unit contact branch, `L_lean_branch_calculus`, reaches 18. Upstream, `audit <node>` returns the ancestor closure together with the credential state of every vertex in it, so we can see whether the route to a theorem passes through anything uncertified. The longest chain in the graph runs 10 edges, from `L_lean_branch_calculus` through the local and global contact branch, the bound and the uniqueness clause of Theorem C, the strict inequality and the finite-tail estimate, and Theorem A itself, to Corollary A’. At that length inspection stops being reliable and the closure does the work.

The manifest graph and Lean’s. Lean keeps a dependency graph of its own, whose vertices are declarations, and enforces it directly: a Lean proof cannot use a lemma that is not there. The verification work happens there, and the manifest does none of it. The manifest records which printed statements carry which evidence, on which premises. Each Lean credential sits on a vertex of its own beside the paper’s vertex: the paper’s vertex keeps the in-edges of the printed proof, and the Lean vertex those of the formal proof, certified facts included. The printed proof of Theorem E uses its scalar facts as steps, while its Lean vertex lists their certificates as premises, so a change to a certificate shows up in the audit of the Lean result. For the single-unit results the Lean vertices mirror the decomposition of the Lean development.

Degrees carry a different kind of information. The checker’s `status` report flags vertices with no descendants and four or more ancestors, currently 12 of them, as costly but load-bearing for nothing. This is a proportionality diagnostic, not a correctness test: a corollary can be sound and still require a deep chain that supports no later result.

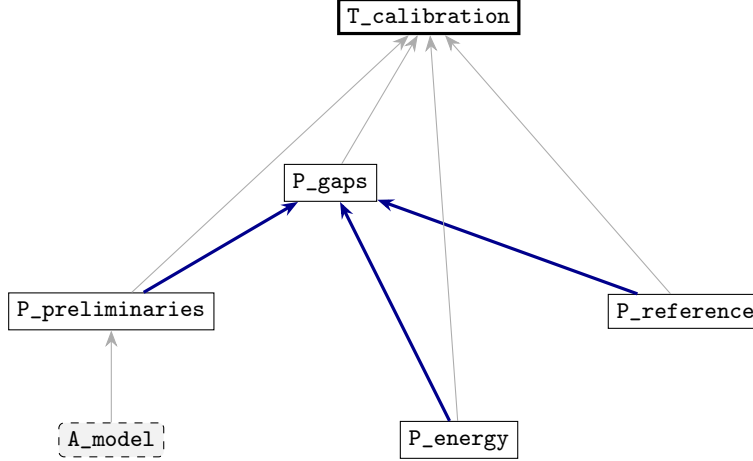


Figure 1: The dependency cone of Theorem C, drawn from the manifest: six vertices and eight edges. Edges run from premise to conclusion, so reading upward follows the paper’s argument. The dashed box is an assumption, which carries no credential by construction, and the thick box is a theorem. The three **bold blue edges** are the in-neighbourhood `P_gaps` is fingerprinted against. Cones elsewhere in the manifest run much wider, with 28 vertices downstream of `A_two_model`.

3 Verification Layers

The claims call for different verification methods.

Deductive claims, which we checked in Lean. Theorems, propositions and lemmas assert implications from stated premises. Of the 29 paper statements in the manifest, 25 have a Lean counterpart, including 9 conditional on certified facts. Coverage ranges from full statements to selected components, as specified below. Section 7 pairs each with its declarations, and Section 9 accounts for the other 4.

Finite computations, which we checked in exact and interval arithmetic. Several claims concern specific numbers: the decimal enclosure of β_* and the data of the finite-tail pairs, the lower bound 73/100 on the two affine boundary families of the curve family, the stationary point of that family and its physical margins, and the rational polygon of Lemma D.7. Exact rational arithmetic, Bernstein expansion and Arb ball arithmetic check them outside Lean (Section 4). The two instances of Theorem D are checked twice: by exact rational arithmetic in the replay script, and inside the Lean kernel by `decide +kernel` on the stored integers.

Symbolic and scalar facts. Identities that expand to zero and first-order inequalities over the reals are certified by exact SymPy expansion, or by the SMT solvers `z3` and `cvc5` separately. Where a Lean proof needs such a fact, the fact enters as a hypothesis (Section 5).

Interpretive statements. Several passages make no mathematical claim, among them the reading of the frontier in the paper’s Section 4 and the comparison with related mechanisms in its Section 14. The numerical evidence offered for Conjecture 13.3 in Section 13.5 is floating-point exploration and carries no credential.

4 The certificates outside Lean

The directory `supplement/` contains the finite instances of Theorem D, the interval data of the two-unit family, and the replay script `verify.py`. The script reads every certificate from its data file and re-evaluates it without search: exact rationals for the instances, the Bernstein trees on the affine boundary families, and Arb balls for β_* , the finite-tail pairs, both covers of the stationary branch, and the root strip. For the files of identities and sign enclosures it confirms the recorded zero residuals and signs rather than re-deriving them. Running it from that directory,

```
python verify.py
```

prints one line per check, among them

```
PASS beta_*: 0.73802433573 < beta_* < 0.73802433574
PASS general instance: ratio < 0.7290804    ratio = 0.7290803900752
PASS stationary_uniform_cover.json: leaves tile t in [1/5, 421/1000]
```

and ends with 37 checks, 0 failed. The scalar facts carried by the hypothesis structures of Section 5 were checked by exact SymPy expansion or by both `z3` and `cvc5`. The repository includes the checking scripts, their receipts, and the shared Python helpers they import. Every Lean field names the check it relies on.

5 Certified facts as Lean hypotheses

For the two-unit results we followed one rule: a fact that an exact, interval or SMT certificate already establishes is not proved again in Lean. It enters the Lean statement as a field of a hypothesis structure. Each field states the certified formula in the notation of the Lean definitions, and its documentation names the paper passage that uses it and the certificate record that establishes it, and says what that certificate checks and no more. There are four structures.

- **PricingScalarCertificates** (12 fields), in `TwoUnit/Pricing/Defs.lean`: four identities and Farkas certificates for the price constraints, checked by exact expansion, and eight scalar inequalities, among them the contraction factor $\bar{b}/(1 + \bar{b})$ of the kernel and four pointwise facts about rounded buyers, each checked by both `z3` and `cvc5`.
- **NodeScalarCertificates** (4 fields), in `TwoUnit/Pricing/FiniteNodes.lean`: a backward step of the node recurrence is the least value above its three bounds and preserves order, kernel summands compare termwise (`z3` and `cvc5`), and the three-way maximum is convex (exact, 27 branch certificates).
- **FamilyNumerics** (10 fields), in `TwoUnit/Family/Certificates.lean`: the value of the small-mass bound (90) at $1/32$, the bound $73/100$ on the two affine boundary families, seven rational constants of the proof of Lemma D.5, the existence, uniqueness and physical margins of the stationary point over the whole interval of trial ratios, its comparison sign at the lower trial ratio, and the rational polygon of Lemma D.7.
- **FamilyIdentities** (52 fields), in the same file: closed-form identities of the curve family, among them the quadratures, the first integral of the Euler equation, the contact densities, the reconstruction formulas, the stationarity reductions, and the price-region algebra of the realization lemma, each an exact symbolic residual or a check by both solvers.

Theorem E assumes `PricingScalarCertificates`, and the finite-node recurrence assumes both pricing structures. Lemma D.7 assumes `FamilyNumerics`, Lemma D.1 assumes `FamilyIdentities`, and Proposition 13.2 with Lemmas D.2 and D.4–D.6 assume both family structures. A conditional credential therefore reads: the printed statement follows in Lean from these fields. The fields are arguments of the theorem, such as `cert : PricingScalarCertificates`, not axioms, so `#print axioms` does not list them and `#check` does.

The two interval covers of the stationary branch are certified only under the premise that the algebraic residual increases in p_f . The four fields that rest on them carry that premise explicitly, and Lean discharges it from the second claim of Lemma D.5, which it proves from certified constants and identities.

6 How to rerun the checks

The toolchain is pinned: Lean `leanprover/lean4:v4.30.0` and Mathlib at revision `c5ea00351c28` (tag `v4.30.0`), the only dependency. From the project directory `research_ideas/probes/fixed_price/lean/` of the repository, after `lake exe cache get` has fetched Mathlib’s compiled files:

```
lake build
```

A clean build of the credentialed development exits zero. It includes the kernel evaluation of the two instances of Theorem D, which takes about four minutes. A single file typechecks on its own with `lake env lean <file>`, writing nothing.

A successful build permits incomplete proofs and additional axioms, so we check axiom dependencies separately. In a file importing `FixedPrice`, the command

```
#print axioms FixedPrice.theoremA
```

should print exactly `[propext, Classical.choice, Quot.sound]`. If `sorryAx` shows up in that list, some proof in the dependency closure is incomplete; any other name means a custom axiom was admitted somewhere. The same holds for every conditional result, whose certified facts sit in its type instead:

```
#check @FixedPrice.TwoUnit.Family.prop_2fam_optimum
```

prints `FamilyNumerics → FamilyIdentities → OptimumStatement`. We also check that the manifest is internally consistent. From the repository root, the command

```
python proof_factgraph/factgraph.py
research_ideas/probes/fixed_price/manuscript/factgraph.toml check
```

recomputes every fingerprint, reports any credential whose statement or premise set has moved since it was issued, and rejects dangling references and cycles in the dependency graph. At this revision it prints ok: 103 nodes, no dangling refs, no cycles.

7 Paper statements and their Lean counterparts

The table pairs each numbered result with its available Lean counterpart, in the order of the paper. Where a row is only partial or conditional coverage, the row says so explicitly; the manifest, rather

than this table, decides whether the complete paper node carries a credential. File paths are relative to `lean/FixedPrice/`, and names relative to the namespace of their file.

The repository also includes `statement_correspondence.py` and its generated TeX/Lean excerpts. The script extracts the table's labels, locates their declarations, and expands the named statement abbreviations and certificate structures. It detects missing names and incorrect file locations. Agreement of quantifiers, assumptions, definitions and equality cases still requires a semantic review; the script issues no equivalence certificate. The scope differences below remain part of that review.

The repository also includes `statement_correspondence.py` and its generated TeX/Lean excerpts. The script extracts the table's labels, locates their declarations, and expands the named statement abbreviations and certificate structures. It detects missing names and incorrect file locations. Agreement of quantifiers, assumptions, definitions and equality cases still requires a semantic review; the script issues no equivalence certificate. The scope differences below remain part of that review.

Paper statement	Lean declaration	File
Sec. 3, scalar curve	(7) and the monotonicity of S and τ : <code>hasDerivAt_optimalValue</code> , <code>optimalValue_strictAntiOn</code> , <code>initialState_strictMonoOn</code> ; limits (8): <code>optimalValue_tendsto_left</code> and its three companions; S onto $(1, \infty)$: <code>existsUnique_optimalValue_eq</code> ; the arctangent form of $\mathcal{I}$, the best price and the randomization clause outside Lean	<code>ScalarCurve.lean</code> , <code>FrontierCurve.lean</code>
Thm. A thm:exact Thm. B thm:frontier	<code>theoremA</code> : every pair above β_* , bounded pairs below $\beta_* + \varepsilon$, and β_* as greatest lower bound; the root C_* : <code>existsUnique_root</code> ; the decimal value of β_* outside Lean <code>theoremB_affine</code> , <code>frontierDelta_eq</code> for (11); boundedness of the refuting and approaching pairs is not in the exported types; <code>frontierDelta_strictMonoOn</code> , <code>frontierDelta_strictConvexOn</code> , <code>frontierDelta_deriv_image</code> ; <code>kappaOf_strictMonoOn</code> , <code>existsUnique_kappaOf_eq</code> for (12); <code>theoremB_conditional</code> , <code>condRatio_eq</code> for (13); <code>theoremB_conjugate</code> for (14)	<code>TheoremA.lean</code> , <code>Strict.lean</code> , <code>ScalarCurve.lean</code> <code>FrontierAffine.lean</code> , <code>FrontierConvex.lean</code> , <code>FrontierCurve.lean</code> , <code>FrontierConditional.lean</code>
Cor. B' cor:asymptotics Lem. 5.1 lem:energy	<code>frontierDelta_isEquivalent_zero</code> , <code>frontierDelta_isEquivalent_one</code> , <code>condRatio_isEquivalent_zero</code> , <code>condRatio_isEquivalent_top</code> <code>reciprocalState_hasDerivAt_at_tailTime</code> , <code>reciprocalState_bounds</code> , <code>reciprocalState_line_obstacle</code> , <code>reciprocal_energy_identity_extended</code> ; original-time form <code>objective_originalTime_identity</code>	<code>Asymptotics.lean</code> <code>Reciprocal.lean</code> , <code>Controls.lean</code>
Lem. 6.1 lem:reference	component coverage only: <code>contact_branch_local_contact_branch_exists</code> , <code>hasBranchRoot_of_le</code> , <code>hasBranchRoot_eventually</code> ; every endpoint reached, <code>exists_branch_point</code> ; (24) by <code>IsBranchRoot.contact_line</code> , <code>IsBranchRoot.contact_arc</code> , <code>IsBranchRoot.contact_cap</code> ; energy <code>IsBranchRoot.logEnergy_refLog</code> ; the parametrization by $\tilde{C}(d)$, the monotone bijection $C \mapsto x(C)$ and the C^1 regularity of P_x not stated	<code>BranchRegularity.lean</code> , <code>GlobalBranch.lean</code> , <code>Coverage.lean</code> , <code>ReferenceCurve.lean</code> , <code>ReferenceEnergy.lean</code>
Lem. 7.1 lem:gaps	(G1): <code>logEnergy_gap_identity_weight</code> , <code>logEnergy_le_of_weight_calibration</code> , <code>eqOn_zero_of_weight_calibration</code> ; (G2): <code>hasDerivAt_endpointValue</code> , <code>physicalEndpoint_calibration</code> , <code>endpointValue_branch_le</code>	<code>CalibrationAE.lean</code> , <code>EndpointDerivative.lean</code> , <code>PhysicalBranch.lean</code> , <code>GlobalBranch.lean</code>
Thm. C thm:calibration	<code>theoremC</code> : bound, attainment by (34), and uniqueness almost everywhere; components <code>objective_le_optimalValue</code> , <code>maximizingControl_attains</code> , <code>objective_eq_optimalValue_iff</code> ; inputs to (G) for $C > 0$; its $C = 0$ endpoint identity not stated	<code>TheoremC.lean</code> , <code>UpperBound.lean</code> , <code>Attainment.lean</code> , <code>Uniqueness.lean</code>
Prop. 8.1 prop:nonexponential	<code>proposition_nonexponential</code> , without the condition $h' > 0$	<code>NoExponential.lean</code>
Lem. 9.1 lem:pricing	<code>exists_price_of_variational_bound</code> , for every $\beta \geq 0$	<code>PricingLemma.lean</code>
Lem. 9.2 lem:hard	<code>HardPairData.sellerMean_eq</code> , <code>HardPairData.gainsFromTrade_eq</code> ; $\Gamma_{\max, \eta}$ and (44) as upper bounds: <code>HardPairData.gain_le</code> , <code>HardPairData.welfare_gap_le</code>	<code>HardBuyer.lean</code> , <code>HardSeller.lean</code> , <code>HardPair.lean</code>
Lem. 12.1 lem:smoothing	<code>lemma_smoothing</code> , in ε - δ form, without the full-support clause	<code>Smoothing.lean</code>
Prop. 12.2 lem:representation	<code>DSICOn.representation</code> , <code>DSICOn.mechWelfare_le</code> , for a class containing the paper's	<code>Mechanism.lean</code> , <code>MechanismWelfare.lean</code>
Cor. A' cor:dsic	<code>corollary_dsic</code> , with <code>dsic_guarantee</code> and <code>dsic_ceiling</code> , for a class containing the paper's	<code>CorollaryDSIC.lean</code> , <code>DSICClass.lean</code>
Thm. D thm:two-separation	<code>TwoUnit.theoremD_general</code> , <code>TwoUnit.theoremD_symmetric</code> , by kernel evaluation; the comparison with β_* uses the interval enclosure outside Lean	<code>TwoUnit/TheoremD.lean</code> , <code>TwoUnit/Checks.lean</code>

Paper statement	Lean declaration	File
Prop. 13.1 prop:two-tail Thm. E thm:two-pricing	analytic proof, outside Lean theoremE_part_i_proof, theoremE_part_ii_proof, theoremE_part_iii_proof, theoremE_part_iii_welfare_proof, theoremE_part_iv_proof, theoremE_part_iv_sup_proof; independence of $\bar{b}$: pricingValue_eq_of_bound_le_proof; each assuming PricingScalarCertificates	manifest node P_two_tail TwoUnit/Pricing/Operator.lean, TwoUnit/Pricing/Attainment.lean, TwoUnit/Pricing/PriceTarget.lean, TwoUnit/Pricing/Rounding.lean TwoUnit/Pricing/PriceTarget.lean
Text after Thm. E	the realized tail density: realizedPrice_spec_proof, assuming PricingScalarCertificates; the single-atom alternative not stated	
Prop. 13.2 prop:2fam-optimum Conj. 13.3 conj:two-exact	prop_2fam_optimum and self_consistency, assuming FamilyNumerics and FamilyIdentities open; its reduction to the budget bound (66) is analytic, outside Lean	TwoUnit/Family/Final.lean, TwoUnit/Family/Assembly.lean manifest node P_two_outer_sufficiency
App. B.5 app:two-recurrence	nine exports in namespace NodeData, among them nodeMass_backward_proof for (82), nodeMass_isLeastSolution_proof, nodeValue_convex_piecewiseAffine_proof, sellerPotential_hasDerivAt_between_nodes_proof, sellerPotential_jump_at_node_proof; assuming PricingScalarCertificates and NodeScalarCertificates	TwoUnit/Pricing/FiniteNodesProofs.lean
Prop. C.1 prop:finite App. C.2 app:nonconcavity	proposition_finiteTail, with the strict inequality $0 < r_\eta - \beta_*$; (84) by objective_liftControl_sub_le; the decimal bounds outside Lean formula (85) analytic, gap 11/1728 in exact arithmetic; no Lean declaration	TheoremA.lean, Sharpness.lean, Perturbation.lean manifest nodes P_nonconcavity, L_jensen_exact
Lem. D.1 lem:2fam-realization	lem_2fam_realization, with weak convergence of the laws; assuming FamilyIdentities	TwoUnit/Family/Realization.lean, TwoUnit/Family/Final.lean
Lem. D.2 lem:2fam-compact	lem_2fam_compact; assuming both family structures	TwoUnit/Family/Compactness.lean, TwoUnit/Family/Final.lean
Lem. D.3 lem:2fam-shape	shape_holds, with no hypothesis	TwoUnit/Family/Shape.lean
Lem. D.4 lem:2fam-boundary	lem_2fam_boundary; assuming both family structures	TwoUnit/Family/Boundary.lean, TwoUnit/Family/Final.lean
Lem. D.5 lem:2fam-domain	lem_2fam_domain; assuming both family structures	TwoUnit/Family/Domain.lean, TwoUnit/Family/Final.lean
Lem. D.6 lem:2fam-cover	lem_2fam_cover, cover_atMostOne; assuming both family structures	TwoUnit/Family/Stationary.lean, TwoUnit/Family/Reconstruction.lean, TwoUnit/Family/Final.lean
Lem. D.7 lem:2fam-witness	lem_2fam_witness; assuming FamilyNumerics	TwoUnit/Family/Witness.lean, TwoUnit/Family/WitnessPoly.lean, TwoUnit/Family/Final.lean
Prop. D.8 prop:2fam-separate-responses	analytic proof with recorded symbolic identities and sign enclosures, outside Lean	manifest node P_two_reference_responses
App. E app:two-finite	the instance of Thm. D, as data generated from the stored exact instance; its construction from the polygon is not needed for the claim	TwoUnit/GeneralInstanceData.lean

8 Where the Lean statement and the printed one differ

The following scope differences matter when reading the credentials.

The dominant-strategy mechanism class. Proposition 12.2 and Corollary A' assume individual rationality and strong budget balance in every realization. The Lean class assumes only their expected consequence $s\varphi \leq \Pi \leq b\varphi$ on the report interval, and no measurability of φ . It contains the paper's class, so the ceiling of Corollary A' is proved for more mechanisms than printed, which makes it stronger; since posted prices lie in both classes, the value β_* holds for every class between the posted prices and the formal one. The representation is stated through a monotone function F with values in $[0, 1]$ rather than the measure $\nu = dF$, with $\Pi(s, b) = b(F(b) - F(s)) - \int_s^b (F(t) - F(s)) dt$, and its exceptional set is countable in s rather than Lebesgue-null in (s, b) .

Smoothing and hard pairs. Lemma 12.1 is stated in ε - δ form: for every $\eta > 0$ an absolutely continuous pair comes within η in first-best welfare and exceeds the original best posted-price welfare by at most η at every price. Taking $\eta = 1/n$ gives the printed sequence. The approximations lie almost surely in $(0, \Lambda)$, which is what Proposition 12.2 needs; full support is not claimed, and the extra uniform mixture that provides it in the paper is omitted. For Lemma 9.2, Lean proves the identities for M_η and G_η in (43), and for $\Gamma_{\max, \eta}$ only the upper bound $\Gamma(z) \leq d + \eta d^2 T_\eta$ at every price, so (44) holds as an inequality at $\beta = 1/(1+d)$. Every use of the lemma in the paper needs only these upper bounds, and the Lean proofs of Theorems A and B, Proposition C.1 and Corollary A' use them in this form.

Bounded witnesses for the frontier. The exported statements `theoremB_affine` and `theoremB_conditional` establish the sharp frontier and the conditional infimum. Their types do not assert the bounded support of the refuting pairs or of the approaching pairs with exactly $G/M = \kappa$ required by Theorem B. The proofs use the explicit hard pairs, with a finite seller atom for the conditional construction; the bounded-support conclusions have not been exported as Lean statements.

Statements proved in stronger form. Lemma 9.1 holds for every $\beta \geq 0$, not only $0 < \beta < 1$. Proposition 8.1 needs neither the condition $h' > 0$ nor $\tau(C) = d$. Proposition C.1 holds with the strict inequality $0 < r_\eta - \beta_*$.

The reference family of Lemma 6.1. Lean proves that the curvatures with a physical contact root form an open, downward-closed set on which the root is unique, and that every endpoint in $(0, 1/d)$ is reached; it organizes the branch by the endpoint denominator $D_C(Y)$ rather than by $\bar{C}(d)$. For every branch point it constructs P_x , proves P_x and the weight $a(\log P_x)'$ Lipschitz, which gives a multiplier Q_x without atoms, and proves (24) on each piece. The monotone bijection $C \mapsto x(C)$ onto $(0, 1/d)$, the bound $\bar{C}(d) > 1/4$, and the continuous differentiability of P_x are not stated. The formal proof of Theorem C uses none of them.

Horizons and composite identities. The integrals over $(0, \infty)$ in (E) and (G1) are formalized on $[0, R]$ for every R beyond the tail time $A(0)$ and the cap point a_a ; both integrands vanish beyond these points, so the two forms agree. For $C > 0$, the identity (G) of Theorem C is recorded through its inputs (E), (G1) and (G2), without a combined declaration. At $C = 0$, the endpoint-gap integral identity is not formalized: Lean handles this case directly by $\mathcal{J}_d(h) \leq 1$. The bound, attainment

and almost-everywhere uniqueness are fully stated and proved. Theorem A uses $\mathcal{I}$ as the integral (3); its arctangent form is a fact of the paper’s Section 3 that Lean does not state.

The two-unit statements. The Lean statements of Theorem E, Proposition 13.2 and its lemmas match the printed ones apart from their certified-fact hypotheses. Lean represents a curve of the class (64) as a function on the real line and compares curves on $[c_f, \xi_0]$, the interval on which the class is defined. Theorem D is stated for the two explicit instances. The general infimum r_2^* is defined as `r2star` in `TwoUnit/Family/GeneralModel.lean`; the comparison applying that infimum to the instances of Theorem D is not exported as a Lean statement. The paper obtains $r_2^* < 0.7290804$ by this comparison.

9 Scope Outside Lean

The 4 paper statements without a Lean counterpart, and the parts of others that Lean does not state, fall into four groups.

Analytic arguments. Proposition 13.1, which reduces r_2^* to admissible body pairs with a common escaping tail, and Proposition D.8, which checks the comparison behind Conjecture 13.3 from each side at the family minimizer, keep their analytic proofs; the second also uses recorded symbolic identities and sign enclosures. So does the reduction of the conjecture to the budget bound (66).

Facts from the paper’s Section 3. The arctangent form (5) of $\mathcal{I}$, the existence of a price attaining the maximal gains from trade, and the fact that randomized prices do no better are not stated in Lean. Lean states each guarantee with an explicit price and each single-unit ratio through the supremum over prices, so none of its results needs a best price to exist.

Finite and interval checks. The decimal values in the paper, the two-step formula (85) with its gap 11/1728 in Appendix C.2, and every field of the four hypothesis structures of Section 5 are certified by exact, interval or SMT arithmetic. These checks have their own manifest layers and do not receive Lean tags.

Open endpoints. Conjecture 13.3, that $r_2^* = r_{\text{fam}}$, is open, and the manifest records its reduction separately. The numerical evidence for it is floating-point exploration.

10 Findings from Formalization

Formalization identified the following gaps and overstatements. None of them changes a printed theorem, proposition or lemma, and the printed proofs and definitions incorporate the corresponding repairs.

1. The proof of Theorem E used, without proof, that $\mathcal{J}_d^{(2)}$ does not depend on the interval $[0, \bar{b}]$ carrying the buyers, and the welfare sentence of part (iii) had no argument. Appendix B.4 proves both, and Lean states the first as a separate declaration. The same appendix makes explicit that rounded buyer bodies stay ordered, the monotone extensions on which Helly’s selection theorem acts, and the space of bounded Borel functions on which the operator contracts.

2. The text after Theorem E said that optimizing the compensation can give several price distributions. The argument supports a weaker statement: the realized price is not the only one, because its tail mass may instead sit in one atom at $\bar{b} + \varpi_{d,\vartheta}(\bar{b})/d$. The text says that.
3. In the finite-node recurrence of Appendix B.5, the printed derivative of the seller potential holds between consecutive nodes; at a node carrying buyer mass but no price atom it is only a one-sided derivative. The appendix states the derivative on the open cells and the jump at each node, as Lean does, and gives the reason for each node inequality.
4. The proof of Proposition 13.2 used, for interior stationary curves, the connection equation (100) and the order $q_{r,f} < v_f$ without deriving them. Its first step derives both. Feasible parameter triples, convergence of instances, outer derivatives, and the algebraic branch were used before being defined; each is defined where it is introduced.
5. The proof of Lemma D.2 asserted without argument that the small-mass bound (90) increases on $(0, 1/32]$, and its interval evaluation covers only the value at $1/32$. The proof gives the derivative, and Lean proves the monotonicity separately.

11 Scope of a Machine Credential

First, a credential says a Lean statement follows from Mathlib and the standard axioms, and a conditional credential says it follows from the named certified facts as well. A conditional credential is therefore only as strong as the certificates it assumes, which rest on SymPy, Arb, z3 and cvc5 rather than on Lean’s kernel. Neither kind says that the Lean statement is a faithful rendering of the English one. The manifest makes that translation auditable by storing, for every Lean vertex, the English statement, the premise set, the credential layer, and the discharging declarations.

Second, none of this speaks to whether the modelling assumptions are the right ones: independent private values, a deterministic posted price with the inclusive trading rule, full prior information, and for two units the increasing submodular model. The numerical evidence for Conjecture 13.3 and the figures are outside the formal development.

Third, the proofs in the artifact are independent reconstructions rather than transcriptions. Where we prove the same statement by a different route, the appendix proof is the one to read for intuition and the Lean proof the one that has been checked. The uniqueness clause of Theorem C avoids a step of the printed proof: two maximizers share their reciprocal state, and the normalization $\int_0^{A(t)} p^{-2} da = 1 - t$ then pins the tail time and hence the control. Proposition 12.2 is proved pointwise, through a rectangle identity and repeated subdivision, instead of through distributions. The shape lemma, Lemma D.3, goes through a relaxed obstacle problem for $\sqrt{\xi}(\log \mathcal{P})'$, which makes the global comparison (93) unnecessary.